Processador MIPS Monociclo com Jumps e Algoritmo Sort

Segunda Avaliação: Implementação RTL, Bateria de Testbenches e Síntese em FPGA

José Dercy Vieira da Silva Filho
Vinicius Menezes
Isabela Furtado

Faculdade de Engenharia Elétrica e da Computação – FEEC
Universidade Federal do Amazonas – UFAM

Arquitetura de Sistemas Digitais e Laboratório

Objetivos da Segunda Entrega

Escopo do Projeto

Implementação e validação completa do processador **MIPS Monociclo de 32 bits** em Verilog HDL, sintetizável para a FPGA Intel MAX 10 (DE10-Lite), com foco na **verificação funcional exaustiva**.

Pilares Essenciais desta Entrega:

- **Caminho de Dados com Jumps:** Suporte a desvios condicionais (`beq`, `bne`) e incondicionais (`j`, `jal`, `jr`).
- **Validação por Testbenches (Rubrica 3 – Foco Principal):** Testes unitários para **todos** os módulos e teste de integração Top-Level executando o algoritmo real de ordenação (`sort.bin`).
- **Automação de Testes:** Bateria de regressão completa via Makefile e scripts bash (`make test`).
- **Síntese e Prototipagem em FPGA:** Análise, elaboração e bitstream gerados no Intel

Visão Geral da Arquitetura MIPS Monociclo

O processador adota o **modelo Harvard** (barramentos separados para código e dados), estruturado em 4 subsistemas principais:

1. Caminho de Dados (Datapath)

Registrador PC, somadores paralelos (PC+4 e Branch), extensores de sinal, multiplexadores em cascata, Banco de Registradores e ULA de 32 bits.

2. Unidade de Controle Integrada

Decodificação combinacional de opcode e `funct`, orquestrando todos os sinais de controle em um único ciclo de clock.

3. Memória de Instruções (ROM)

Armazenamento estático de 256 palavras de 32 bits inicializado via `$readmemb` com o firmware compilado (`sort.bin`).

4. Memória de Dados (RAM)

Memória síncrona de 256 palavras para leitura (`lw`) e escrita (`sw`), onde o vetor desordenado é processado e ordenado *in-place*.

Caminho de Dados: Suporte Completo a Jumps

Para garantir chamadas e retornos de procedimentos (`jal` e `jr`) e desvios no mesmo ciclo, foi projetada uma **cascata de multiplexadores para o próximo PC**:

Lógica de Decisão do PC:

1. **Branch (`beq` / `bne`):**

   $\text{pc_src} = (\text{branch} \,\&\, \text{zero}) \mid (\text{bne} \,\&\, \sim \text{zero})$

   Calcula: $(\text{PC} + 4) + (\text{offset} \ll 2)$.

2. **Jump Direto / JAL (`j` / `jal`):**

   $\text{PC} \leftarrow \{\text{PC+4}[31:28],\ \text{target}[25:0],\ \text{2'b00}\}$

3. **Jump Register (`jr`):**

   $\text{PC} \leftarrow \text{Reg}[rs] \quad (\text{normalmente Reg}[\$ra])$

Suporte a Funções e Link

- **JAL (Chamada):** O endereço de retorno $\text{PC} + 4$ é gravado no registrador **\$ra (\$31)** via multiplexador `mem_to_reg`.
- **JR (Retorno):** Desvia a execução de volta ao chamador.
- **Trava de Segurança:** O sinal `reg_write` é mascarado como $\text{reg_write} \,\&\, \sim \text{jr}$, evitando corrupção de registradores no retorno.

Síntese e Prototipagem em FPGA (Rubrica 1)

Configuração no Intel Quartus Prime

- **Dispositivo Alvo:** FPGA **Intel MAX 10** (`10M50DAF484C7G`) na placa **Terasic DE10-Lite**.
- **Automação via Makefile:** Alvos `make quartus` (análise/elaboração) e `make quartus-build` (síntese, fitter e bitstream).

Resultados de Compilação

- **Análise e Elaboração: 0 erros**.
- **Módulos RTL:** 15 módulos HDL reconhecidos e sintetizados com sucesso.
- **RTL Viewer:** Esquemático livre de latches espúrios e barramentos desconectados.

Fluxo de Programação JTAG

- `jtagconfig`: Reconhecimento do gravador USB-Blaster.
- `quartus_pgm`: Gravação do arquivo `.sof` na SRAM do FPGA com comando único (`make quartus-pgm`).

Estratégia de Validação e Regressão (Rubrica 3 – Foco Central)

Filosofia de Verificação da Equipe

Nenhum módulo é integrado ao Top-Level sem antes ser exaustivamente validado por um testbench unitário dedicado, com verificação automatizada via assertivas (`if/else` com contadores de falhas).

Resumo da Bateria Completa de Regressão (`make test`):

Módulo Testado	Arquivo de Testbench	Qtd. Testes
Unidade Lógica e Aritmética	`tb/tb_alu.v`	11 cenários
Datapath Auxiliar (Adders, MUX, Extensor)	`tb/tb_datapath_aux.v`	8 cenários
Banco de Registradores (RegFile)	`tb/tb_reg_file.v`	6 cenários
Unidade de Controle Integrada	`tb/tb_control_unit.v`	13 instruções
Memória de Instruções (ROM)	`tb/tb_instr_mem.v`	5 palavras
Memória de Dados (RAM)	`tb/tb_data_mem.v`	4 cenários
Top-Level MIPS Monociclo (Sort Real)	`tb/tb_mips_monociclo_top.v`	5000+ ciclos
Total da Suíte de Testes	**7 Testbenches Distintos**	**47+ Verificações**

Validação Unitária: Datapath Auxiliar e Registradores

`tb_datapath_aux.v` (Componentes Auxiliares)

Valida os blocos isolados do caminho de dados:

- **PC Register:** Atualização síncrona com reset ativo em nível alto.
- **Somadores Paralelos:** Cálculo de PC $+ 4$ e endereço de branch com shift (offset $\ll 2$).
- **Extensor de Sinal:** Imediatos positivos e negativos (ex: extensão de `0xFFFF` para `0xFFFFFFFF`).
- **Multiplexadores 2:1 e 3:1:** Seleção determinística de rotas de dados.

Status: **Todos os 8 testes aprovados com 0**

`tb_reg_file.v` (Banco de Registradores)

Verifica as regras estritas da arquitetura MIPS:

- **Invariante $zero ($0):** Leitura fixa em 0 e bloqueio garantido contra qualquer escrita.
- **Leitura Assíncrona Dupla:** Leitura combinacional simultânea de duas portas (`rs` e `rt`).
- **Escrita Síncrona:** Escrita ativada por `reg_write` sincronizada na borda de subida do clock.
- **Registrador $ra ($31):** Escrita e leitura validadas para suporte a `jal`.

Status: **Todos os 6 testes aprovados com 0**

Validação Unitária: ULA e Unidade de Controle

`tb_alu.v` (Unidade Lógica e Aritmética)

11 cenários de estresse numérico e lógico:

- **Lógicas:** `AND`, `OR`, `NOR`.
- **Aritmética com Sinal:** Soma e subtração com operandos positivos e negativos (ex: $10 + (-3) = 7$).
- **Zero Flag:** Ativação automática quando o resultado é nulo ($\sim |$ alu_result).
- **SLT com `$signed`:** Validações com números com e sem sinal (ex: $-10 < 2 \rightarrow 1$).
- **Operação SLL:** Deslocamento à esquerda por `shamt` ($B \ll$ shamt).

`tb_control_unit.v` (Unidade de Controle)

Decodificação exaustiva de todas as instruções:

- **Tipo-R:** `add`, `sub`, `and`, `or`, `slt`, `nor`, `sll`, `jr`.
- **Tipo-I:** `lw`, `sw`, `addi`, `slti`, `beq`, `bne`.
- **Tipo-J:** `j` e `jal`.
- **Trava Crítica do JR:** Confirmação de que `reg_write = 0` durante a execução de `jr`, impedindo escrita indevida em registrador.

Status: 13 / 13 instruções validadas com 0 erros.

Validação Unitária: Memória de Instruções e Dados

`tb_instr_mem.v` (Memória de Instruções – ROM)

Valida a leitura do código de máquina montado:

- **Alinhamento por Palavra:** Indexação por addr$[9:2]$, ignorando os 2 bits menos significativos (divisão por 4 em hardware).
- **Carga do Firmware (`sort.bin`):** Leitura exata das instruções binárias nos endereços `0x0`, `0x4`, `0x8`, `0xC` e `0x10`.
- **Leitura Combinacional:** Resposta imediata sem latência de ciclo de clock.

Status: Leituras de palavras aprovadas (0 erros).

`tb_data_mem.v` (Memória de Dados – RAM)

Valida a integridade do armazenamento de dados:

- **Escrita Síncrona (`sw`):** Dados gravados com segurança apenas na borda de subida do clock quando `mem_write = 1`.
- **Proteção de Escrita:** Inércia da memória quando `mem_write = 0`.
- **Leitura Protegida (`lw`):** Saída forçada a zero quando `mem_read = 0`, evitando propagação de ruído.
- **Múltiplos Endereços:** Teste com gravações e leituras sequenciais.

Teste Top-Level: O Desafio do Algoritmo Sort

Cenário Real da Disciplina

O teste definitivo da Entrega 2 consistiu em executar o código de máquina do algoritmo de ordenação (`sort.bin`), derivado do assembly MIPS `sort.s`, comprovando a capacidade de ordenar um vetor na memória RAM.

Desafios Técnicos Superados Sem Alterar o Binário Fornecido:

- **Instruções Estendidas:** O `sort.s` demandava `sll` (cálculo de deslocamento do array), `slti` (teste de índice $j < 0$) e `bne` (desvio por diferença). O hardware foi expandido com suporte nativo a essas 3 instruções.
- **Bypass de Endereço Base do MARS:** O montador MARS gera endereços com base em `0x00400000`. Uma máscara em nível de barramento (addr$[9:2]$) mapeia essas referências diretamente nas 256 palavras da ROM.
- **Bootloader de Hardware no Reset:** O código do `sort` é uma função pura (procedimento). Ao ligar o hardware, o registrador de reset inicializa automaticamente:

Resultado da Validação Top-Level: Antes e Depois na RAM

A simulação do processador completo (`tb_mips_monociclo_top.v`) comprovou a execução autônoma de centenas de ciclos de clock, manipulando laços aninhados e realizando chamadas `jal swap` e retornos `jr $ra`.

Estado Inicial da Memória RAM (Tempo 0)

Vetor Desordenado na RAM:

Endereço	Índice	Valor Armazenado
0x00	ram[0]	**88**
0x04	ram[1]	**15**
0x08	ram[2]	**42**
0x0C	ram[3]	**7**
0x10	ram[4]	**99**

Estado Final da Memória RAM (Fim do Sort)

Vetor Perfeitamente Ordenado:

Endereço	Índice	Valor Armazenado
0x00	ram[0]	**7**
0x04	ram[1]	**15**
0x08	ram[2]	**42**
0x0C	ram[3]	**88**
0x10	ram[4]	**99**

Inspeção Temporal de Sinais no GTKWave

Geração e Análise de Formas de Onda (Arquivos `.vcd`)

Todos os testbenches geram dumps VCD em `build/`, visualizáveis via comandos `make waves` e `make waves-top`.

Eventos Críticos Rastreados no Top-Level:

- **Evolução do PC:** Transição contínua $\text{PC} \leftarrow \text{PC} + 4$, interceptada por saltos para rótulos de laço (`for1tst`, `for2tst`).
- **Chamada com Link (`jal`):** Barramento `reg_write_data` recebendo $\text{PC} + 4$ e endereçamento de escrita em `write_reg = 5'd31` ($ra).

Evidências de Escrita na RAM

- **Pulsos de `mem_write`:** Ativação exclusiva nas instruções `sw` da sub-rotina `swap` e salvamento da pilha ($sp).
- **Pilha Operacional:** Ajuste do ponteiro de pilha via `addi $sp, $sp, -20` preservando os registradores salvos ($s0-$s3 e $ra).
- **Sinais da ULA:** Transição alternada da

Conformidade com as Rubricas de Avaliação

Rubrica	Requisito Oficial	Status
Rubrica 1	Configuração do ambiente de desenvolvimento e síntese no Quartus Prime	**100% Concluído**
Rubrica 2	Caminho de dados completo com suporte a Jumps (`j`, `jal`, `jr`)	**100% Concluído**
Rubrica 3	Validação de **todos** os componentes via testbenches individuais e top-level	**100% Concluído**
Rubrica 4	Código 100% parametrizado (`parameter DATA_WIDTH = 32`, etc.)	**100% Concluído**
Rubrica 5	Apresentação em vídeo (YouTube) e relatório técnico padrão IEEE	**100% Concluído**

Conclusão e Encerramento

Síntese dos Resultados Alcançados

- **Processador MIPS Monociclo Operacional:** Implementado em Verilog HDL sintetizável e parametrizado.
- **Suíte de Testes Exaustiva:** 7 testbenches unitários e de integração aprovados com 100% de sucesso.
- **Validação Prática com Algoritmo Real:** Execução autônoma do algoritmo de ordenação `sort.bin`, comprovando leitura, desvios e escrita síncrona na memória RAM.
- **Pronto para o FPGA:** Projeto compilado e sintetizado com 0 erros no Intel Quartus Prime para a placa DE10-Lite.

Obrigado pela atenção!

Equipe: José Dercy Vieira da Silva Filho · Vinicius Menezes · Isabela Furtado
Universidade Federal do Amazonas – UFAM / FEEC